

HANDLING EXCEPTIONS

1.Create a table MESSAGE table with a column Result of type varchar2(1000)

Write a program that prompt for SAL and get the corresponding ENAME from the EMP table. Do not use Explicit Cursors. If the value entered return one row then Insert into MESSAGES table with ENAME|| ' ' || SAL.If the value returned is more than one row then Insert into MESSAGES table as 'More than one employee with a salary<>. If no Value is returned then insert into MESSAGES table as 'NO EMPLOYEE with that salary <>.Any other error insert 'other Error'.Use all NAMED EXCEPTION HANDLERS.

```

SQL> CREATE TABLE MESSAGES
2 (
3   RESULT VARCHAR2(1000)
4 );

Table created.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2   v_sal EMP.SALTYPE;
3   v_ename EMP.ENAMETYPE;
4
5 BEGIN
6   v_sal := &SAL;
7
8   SELECT ename
9   INTO v_ename
10  FROM emp
11  WHERE sal = v_sal;
12
13  INSERT INTO messages
14  VALUES (v_ename || ' ' || v_sal);
15
16  EXCEPTION
17    WHEN NO_DATA_FOUND THEN
18      INSERT INTO messages
19      VALUES ('NO EMPLOYEE with that salary ' || v_sal);
20
21    WHEN TOO_MANY_ROWS THEN
22      INSERT INTO messages
23      VALUES ('More than one employee with salary ' || v_sal);
24
25    WHEN OTHERS THEN
26      INSERT INTO messages
27      VALUES ('Other Error');
28 END;

```

```

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2   v_sal EMP.SALTYPE;
3   v_ename EMP.ENAMETYPE;
4
5 BEGIN
6   v_sal := &SAL;
7
8   SELECT ename
9   INTO v_ename
10  FROM emp
11  WHERE sal = v_sal;
12
13  INSERT INTO messages
14  VALUES (v_ename || ' ' || v_sal);
15
16  EXCEPTION
17    WHEN NO_DATA_FOUND THEN
18      INSERT INTO messages
19      VALUES ('NO EMPLOYEE with that salary ' || v_sal);
20
21    WHEN TOO_MANY_ROWS THEN
22      INSERT INTO messages
23      VALUES ('More than one employee with salary ' || v_sal);
24
25    WHEN OTHERS THEN
26      INSERT INTO messages
27      VALUES ('Other Error');
28 END;
29 /
Enter value for sal: 880
old 6:   v_sal := &SAL;
new 6:   v_sal := 880;

PL/SQL procedure successfully completed.
SQL>

```

DATE:25-07-2026

2. Write a program that will prompt for Empno, ename, sal and insert them into the Emp table. Use Unnamed system defined exception handler.

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2   v_empno EMP.EMPNO%TYPE;
3   v_ename EMP.ENAME%TYPE;
4   v_sal   EMP.SAL%TYPE;
5
6 BEGIN
7   v_empno := &EMPNO;
8   v_ename := &ENAME;
9   v_sal   := &SAL;
10
11  INSERT INTO EMP (EMPNO, ENAME, SAL)
12  VALUES (v_empno, v_ename, v_sal);
13
14  COMMIT;
15
16  DBMS_OUTPUT.PUT_LINE('Employee inserted successfully.');
```

Enter value for empno: 7359
old 7: v_empno := &EMPNO;
new 7: v_empno := 7359;
Enter value for ename: SMITH
old 8: v_ename := &ENAME;
new 8: v_ename := 'SMITH';
Enter value for sal: 800
old 9: v_sal := &SAL;
new 9: v_sal := 800;
Employee inserted successfully.

PL/SQL procedure successfully completed.

SQL> |

3. Write a program that will prompts for a EMPNO and delete the employee from the EMP table. Handle with User Defined Exceptions.

```
new 9: v_sal := 800;  
Employee inserted successfully.
```

PL/SQL procedure successfully completed.

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2   v_empno EMP.EMPNO%TYPE;
3
4   emp_not_found EXCEPTION;
5
6 BEGIN
7   v_empno := &EMPNO;
8
9   DELETE FROM EMP
10  WHERE EMPNO = v_empno;
11
12  IF SQL%ROWCOUNT = 0 THEN
13    RAISE emp_not_found;
14  ELSE
15    COMMIT;
16    DBMS_OUTPUT.PUT_LINE('Employee deleted successfully.');
```

Enter value for empno: 10
old 7: v_empno := &EMPNO;
new 7: v_empno := 10;
Employee number 10 does not exist.

PL/SQL procedure successfully completed.

SQL> |

DATE:25-07-2026

4.Create a stock table with columns PNO,PNAME,RATE AND TR_QTY. Insert some records in this table.Create a PL/SQL block that takes PNO,TR_TYPE and TR_QTY from the Stock Tble.f PNO is found and TR_TYPE is 'R' then update the TR_QTY with Old TR_QTY plus NEW TR_QTY. Handle the user defined exceptions using USER DEFINED EXCEPTION HANDLERS.

```
SQL> CREATE TABLE STOCK
2 (
3     PNO          NUMBER PRIMARY KEY,
4     PNAME        VARCHAR2(30),
5     RATE         NUMBER(5,2),
6     TR_QTY       NUMBER
7 );

Table created.

SQL> INSERT INTO STOCK VALUES (101, 'PEN', 20, 100);

1 row created.

SQL> INSERT INTO STOCK VALUES (102, 'BOOK', 50, 200);

1 row created.

SQL> INSERT INTO STOCK VALUES (103, 'PENCIL', 10, 150);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
2     v_pno          STOCK.PNO%TYPE;
3     v_tr_type      CHAR(1);
4     v_tr_qty       STOCK.TR_QTY%TYPE;
5
6     pno_not_found  EXCEPTION;
7     invalid_tr_type EXCEPTION;
8
```

```
9 BEGIN
10     v_pno := &PNO;
11     v_tr_type := UPPER('&TR_TYPE');
12     v_tr_qty := &TR_QTY;
13
14     -- Check whether PNO exists
15     UPDATE STOCK
16     SET TR_QTY = TR_QTY + v_tr_qty
17     WHERE PNO = v_pno
18     AND v_tr_type = 'R';
19
20     IF SQL%ROWCOUNT = 0 THEN
21         IF v_tr_type <> 'R' THEN
22             RAISE invalid_tr_type;
23         ELSE
24             RAISE pno_not_found;
25         END IF;
26     END IF;
27
28     COMMIT;
29     DBMS_OUTPUT.PUT_LINE('Stock updated successfully.');
```

```
30
31 EXCEPTION
32     WHEN pno_not_found THEN
33         DBMS_OUTPUT.PUT_LINE('Product Number not found.');
```

```
34
35     WHEN invalid_tr_type THEN
36         DBMS_OUTPUT.PUT_LINE('Invalid Transaction Type. Enter R only.');
```

```
37 END;
38 /

Enter value for pno: 101
old 10:   v_pno := &PNO;
new 10:   v_pno := 101;
Enter value for tr_type: R
old 11:   v_tr_type := UPPER('&TR_TYPE');
new 11:   v_tr_type := UPPER('R');
Enter value for tr_qty: 50
old 12:   v_tr_qty := &TR_QTY;
```

DATE:25-07-2026

```
17 WHERE pno = v_pno
18 AND v_tr_type = 'R';
19
20 IF SQL%ROWCOUNT = 0 THEN
21     IF v_tr_type <> 'R' THEN
22         RAISE invalid_tr_type;
23     ELSE
24         RAISE pno_not_found;
25     END IF;
26 END IF;
27
28 COMMIT;
29 DBMS_OUTPUT.PUT_LINE('Stock updated successfully.');
```

```
30
31 EXCEPTION
32 WHEN pno_not_found THEN
33     DBMS_OUTPUT.PUT_LINE('Product Number not found.');
```

```
34
35 WHEN invalid_tr_type THEN
36     DBMS_OUTPUT.PUT_LINE('Invalid Transaction Type. Enter R only.');
```

```
37 END;
38 /
Enter value for pno: 101
old 10:      v_pno := &pno;
new 10:      v_pno := 101;
Enter value for tr_type: R
old 11:      v_tr_type := UPPER('&tr_type');
new 11:      v_tr_type := UPPER('R');
```

```
Enter value for tr_qty: 50
old 12:      v_tr_qty := &tr_qty;
new 12:      v_tr_qty := 50;
Stock updated successfully.

PL/SQL procedure successfully completed.

SQL> |
```